

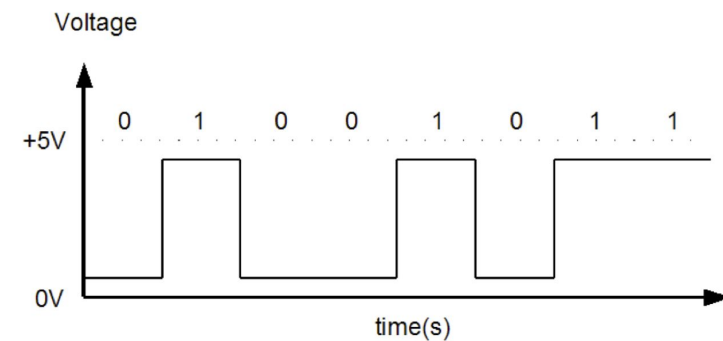
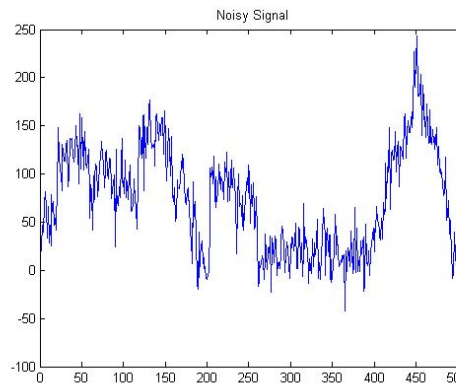
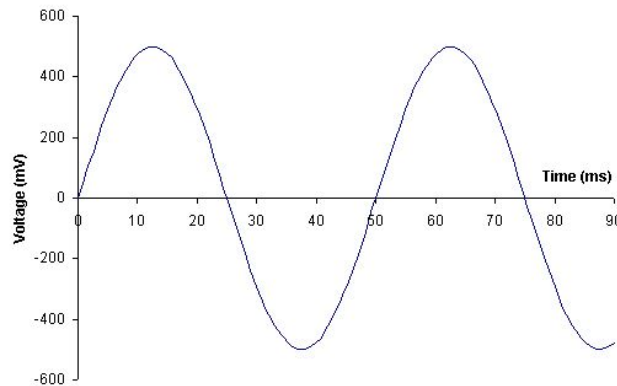
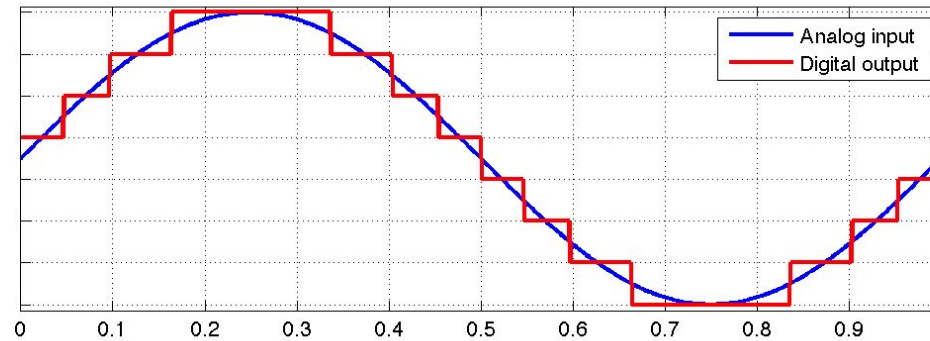


Signal, Timer and Timing

Programming and Development of Embedded Systems

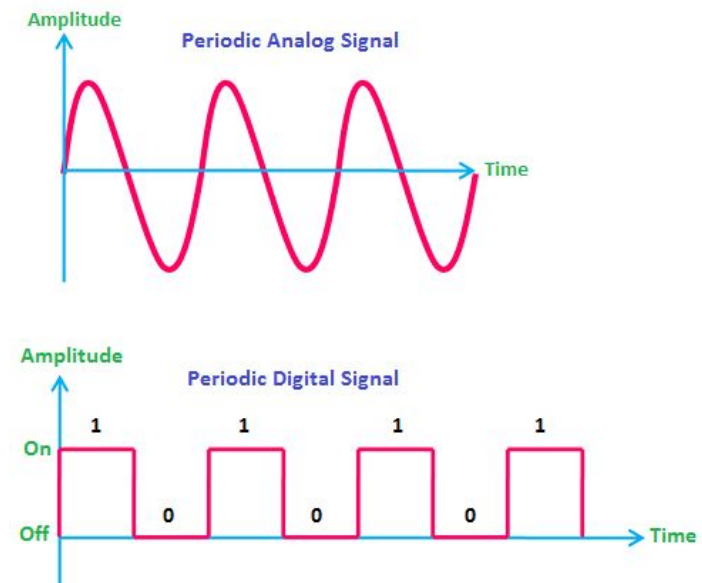
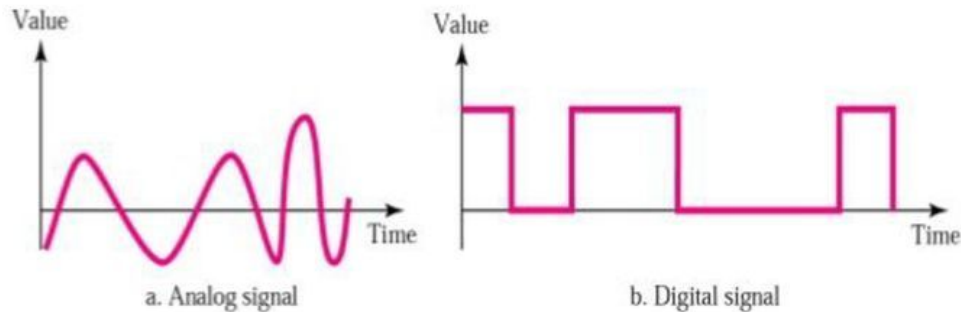
Electrical Signals

- ❖ A Signal is defined as **physical quantities** which contains **information**
- ❖ Signals are passed between devices in order to send and receive information
- ❖ Signals mainly can be classified as
 - Analog signals
 - Digital signals



Periodic and Non-periodic Signals

- ❖ A Periodic signal is a signal which **repeats** itself intervally
 - Examples: Sine, cosine, square, rectangular and triangular signals, and etc.
- ❖ A Non-periodic signal is a signal which **does not repeat** itself after a specific interval
 - Examples: Sound signals from radio and all types of noise signals, and etc.

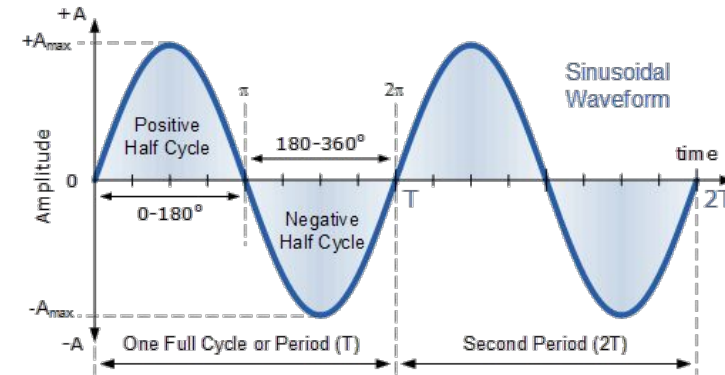


Periodic Signals

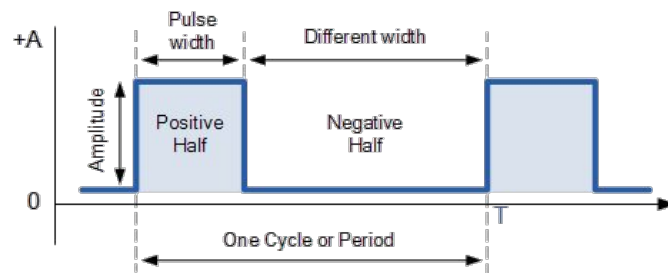
❖ Some Periodic Signal Waveforms

$$\text{Frequency} = \frac{1}{\text{Periodic time}} \quad \text{or} \quad f = \frac{1}{T} \text{ Hz}$$

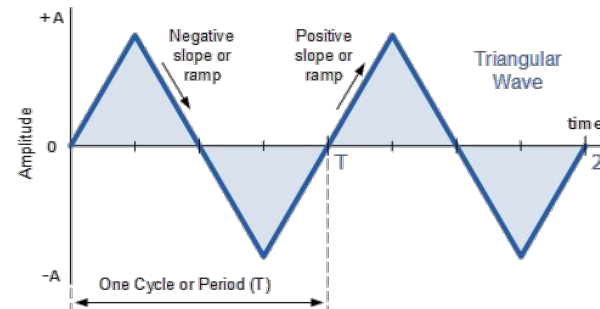
$$\text{Periodic time} = \frac{1}{\text{Frequency}} \quad \text{or} \quad T = \frac{1}{f} \text{ sec}$$



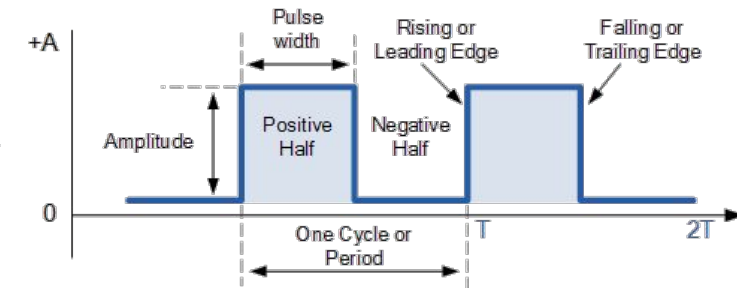
A Sine Wave Waveform



A Rectangular Wave Waveform



A Triangular Waveform

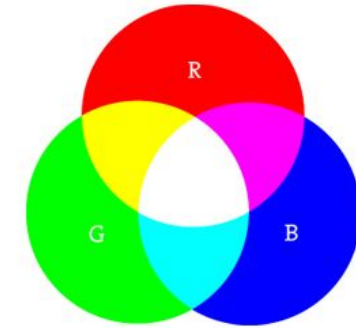


A Square Wave Waveform

PWM Signals

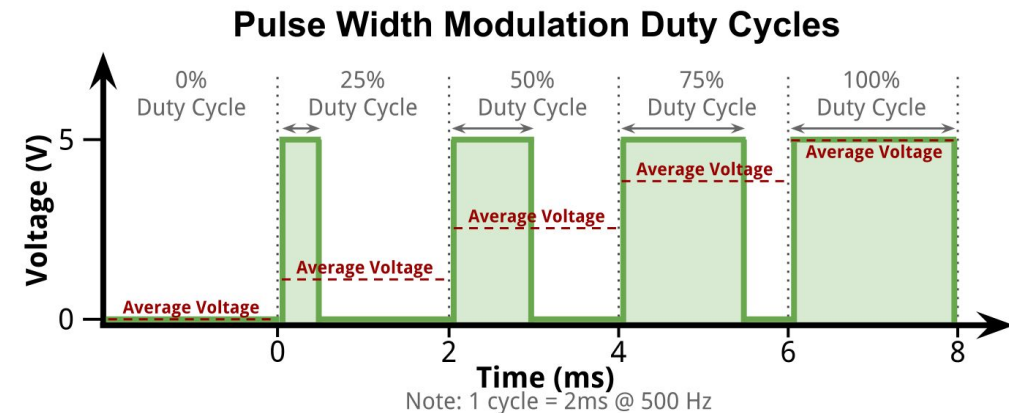
❖ Pulse Width Modulation (PWM)

- Is a fancy term for describing a rectangular digital signal
- Is a technique for getting analog results with digital means
- Is used in a variety of applications like control of
 - The brightness of an LED, the angle of a servo motor, generating clock signal and etc.



❖ Examples

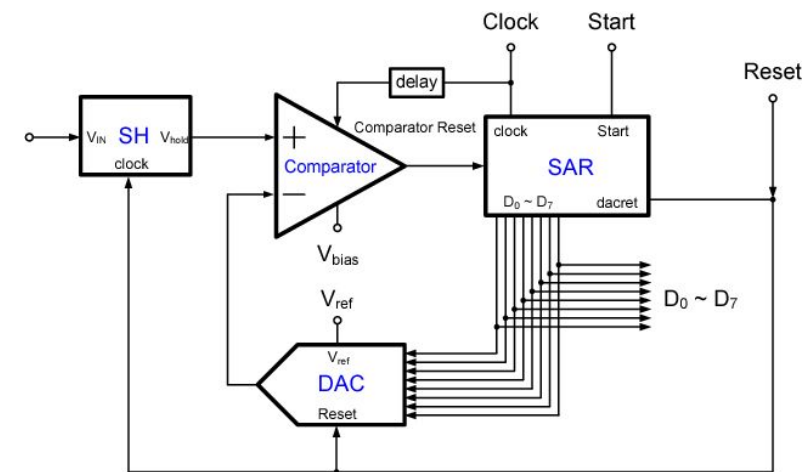
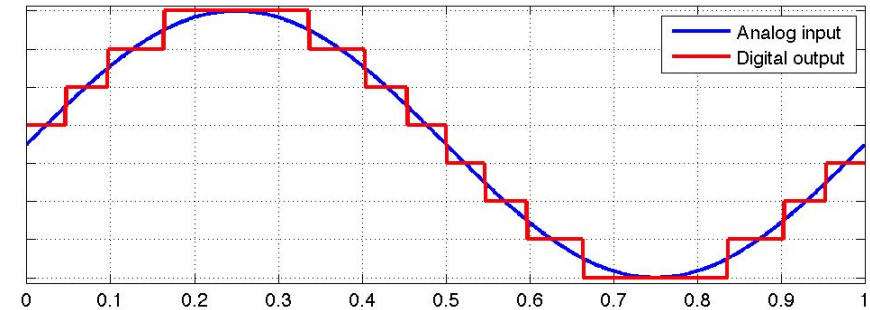
- We can control the brightness of an LED by adjusting the duty cycle
- With an RGB (red green blue) LED, we can control how much of each color we want to have in the mixed colors by dimming them



Duty Cycle describes the proportion of 'on' time to the regular interval or 'period' of time

Analog to Digital Converter

- ❖ ADC Resolution: Number of the output bits
- ❖ Sample Rate: Number of samples per second (Hz)
- ❖ Reference Voltage: System voltage (V_{ref})
- ❖ Voltage Resolution
 - The change in voltage required to guarantee a change in the output code level
 - It is the least significant bit (LSB) voltage
 - $V_{res} = V_{ref} / (2^{\text{ADC Resolution}} - 1)$
- ❖ Digital Value = Analog Voltage / V_{res}



Digital to Analog Converter

❖ DAC Resolution

- Number of the input bits

❖ Reference Voltage

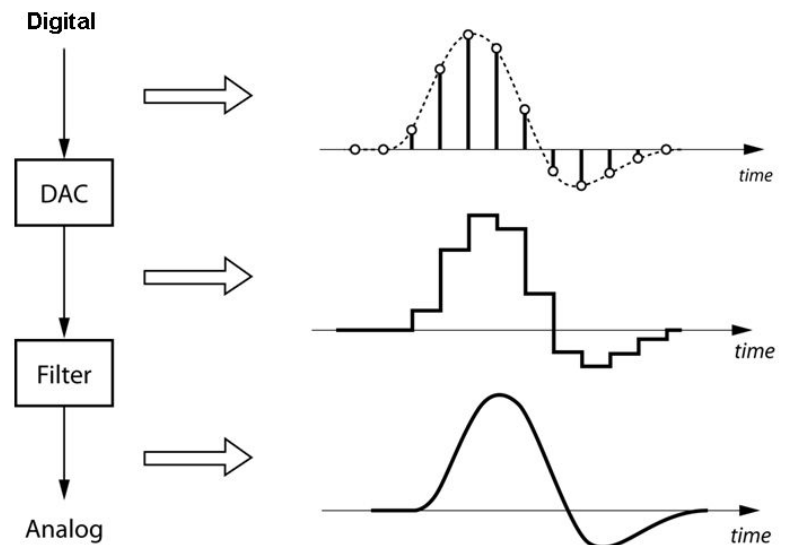
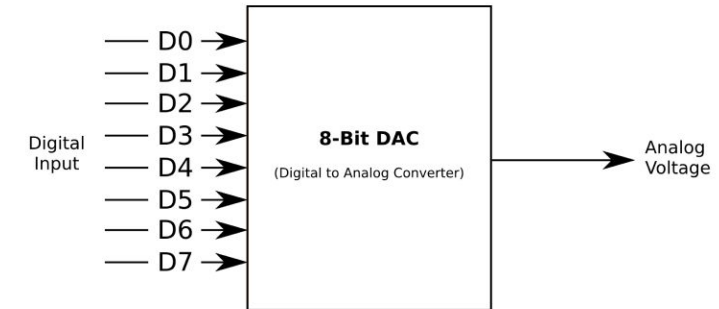
- System voltage (V_{ref})

❖ Voltage Resolution

- The least significant bit (LSB) voltage
- $V_{res} = V_{ref} / (2^{\text{DAC Resolution}} - 1)$

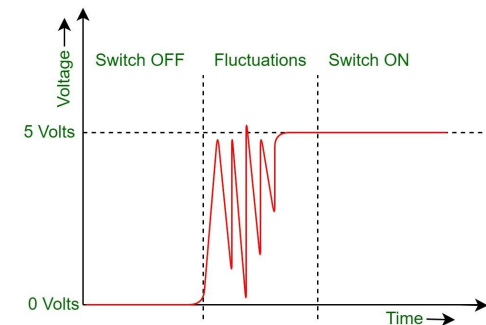
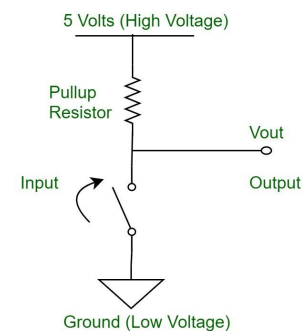
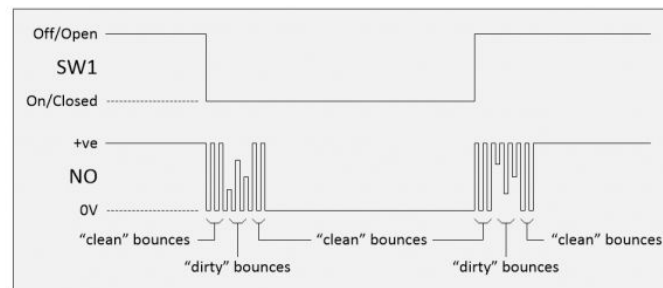
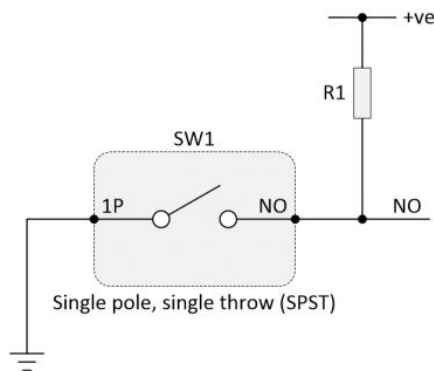
❖ Analog Voltage

- $V_{out} = \text{Digital Value} \times V_{res}$



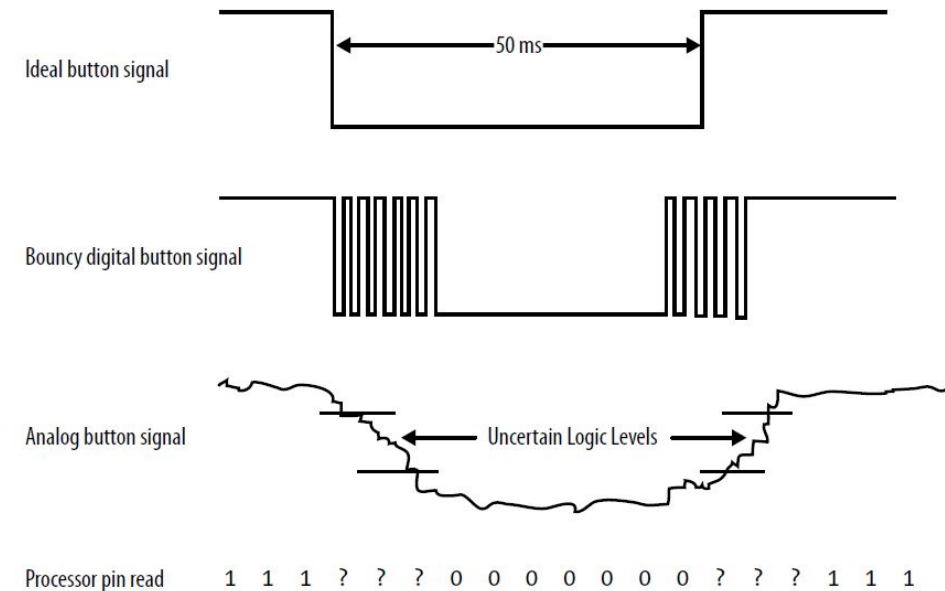
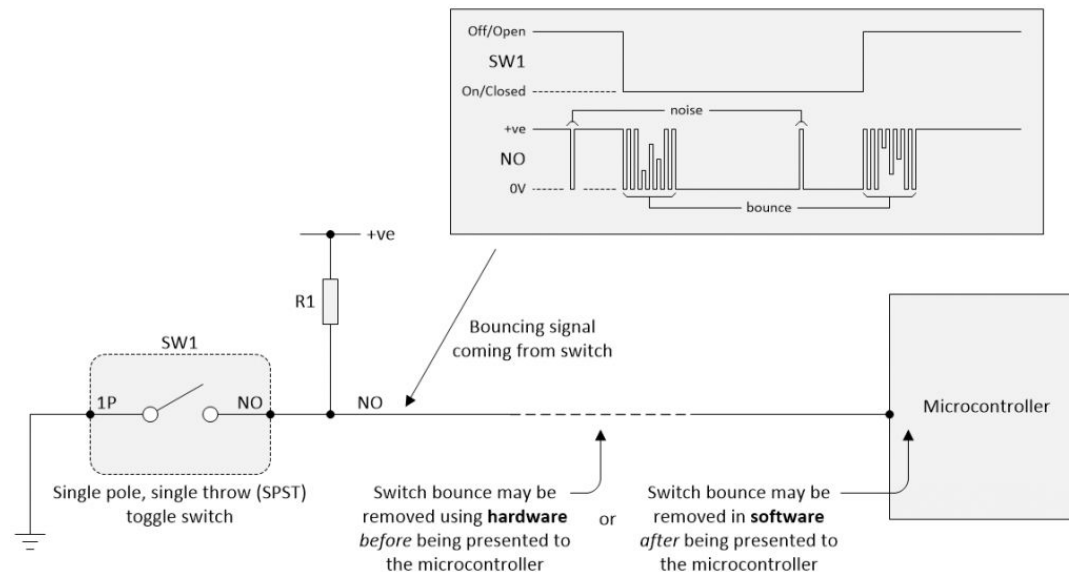
Switches and Contact Bouncing

- ❖ Metal contacts are used in mechanical switches and electromechanical relays
- ❖ Signal bouncing happens when the state of the switch is toggled
- ❖ Mechanical bounce of the contacts which makes and breaks the circuit causes bouncing
- ❖ Contact bouncing happens even if there is no mechanical bounce due to:
 - An inductive effect caused by the switch (the switch acts as an inductor)
 - The transmission line effects in the signal trace
- ❖ Contact bouncing causes multiple states changes per user's action



Switches and Contact Bouncing

- ❖ To the microcontroller the button appears to be pressed multiple times for an extremely short duration, when you think you've just pressed it once.

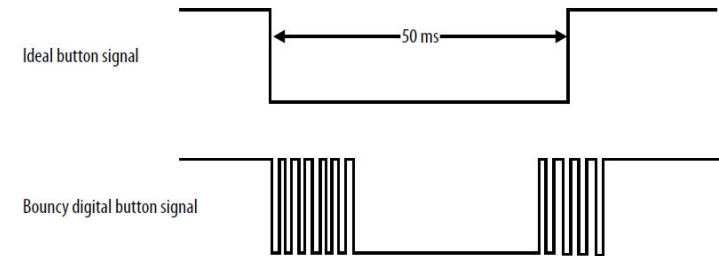


An analog view of what could happen when a button is pressed and only slowly goes into effect. There are parts of the analog signal where the signal is neither high nor low, but somewhere in between. This leads to indeterminate values of the digital signal.

Switches and Contact Debouncing

- ❖ Debouncing is all about making sure that when a switch push or release event happened.
- ❖ Debouncing can be done in hardware or software
- ❖ Software debouncing

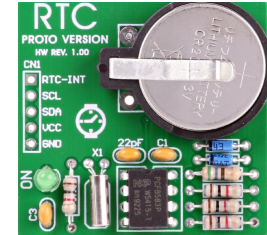
- Read the datasheet of the switch for the period of uncertainty
- To avoid the garbage near the rising and falling edges, you'll need to look for a long period of consistent signal
 - How long? it depends on the switch and how fast you want to respond to the user.
- To debounce the switch, take multiple samples of the pin at a periodic interval several times faster than you'd like to respond. When there have been several consistent samples, alert the rest of the system that the switch has been changed
- For example if a button is hitting 10 times per second and it is down half the time (50 ms), we can sample every 5ms and look for five continued consistent samples



Timer and Timing

❖ Real-Time Clock (RTC)

- A device to keep track of date and time. Usually a dedicated 32.768 kHz crystal timer
 - The same frequency used in **quartz** clocks and watches
 - Oscillators with exactly 2^{15} cycles per second
- RTCs often have an alternate source of power. They can also be powered by a battery.
- RTCs can be external or internal. The POSIX standard time.h can be used.



❖ SysTick Timer: Special timer to keep track of system time.

- In milliseconds and microseconds. Counters are used. The counters will overflow.
- Can be used for timing, measuring time and making delays.

❖ Watchdog Timer (WDT): A Timer used to detect and recover from system malfunctions.

- It monitors the system's operation and ensures that the system is running correctly. If the system becomes unresponsive or hangs, the watchdog timer will reset the system

❖ Programmable Interrupt Timers (PIT)

- Hardware timers which are used to trigger events in regular repeating intervals

Interrupts

- ❖ There are two ways to handle an event
 - Polling; checking for the condition periodically or continuously
 - Interrupt: the processor executes a special piece of code if the event occurs
 - This piece of code is called interrupt handler or interrupt service routine
- ❖ Interrupt
 - Is a hardware feature and many interrupts are available for conditions like
 - Pins changing, data received, timers overflowing, errors and etc.
 - Every interrupt has a flag bit
 - It is set by hardware when the interrupt trigger condition occurs
 - Normally it is automatically reset when the interrupt service routine is called
 - The flag bit is set even if interrupts are not used
 - Polling can read the flag bit to check if the event has occurred

Interrupts ...

- ❖ Every interrupt also has a mask bit to enable/disable interrupts individually
- ❖ Also a global interrupt enable/disable bit for all interrupts that have their mask bits set
- ❖ Prioritized interrupts are possible; a value 0 - 255. **0** for the highest and **255** to the lowest priority
- ❖ Note that not all devices are interrupt-safe. E.g. an SD card.
- ❖ Nested interrupts are possible, but should be avoided by
 - Clearing the global interrupt flag in the beginning of interrupt service routines and
 - Setting the global interrupt flag in the end of the interrupt service routines
- ❖ Keep all interrupt service routines short and simple
 - Interrupts change the timing of programs
- ❖ Shared variables must be declared as **volatile** and be protected

Signals, Time and Timing

❖ Some useful links

- [Electrical Waveforms](#)
- [Analog vs. Digital](#)
- [Pulse Width Modulation](#)
- [Analog to Digital Conversion](#)
- [Digital to Analog and Analog to Digital Conversion](#)
- [Analog to Digital Converter library for the Teensy](#)
- [A Guide to Debouncing](#)
- [Debounce Your Noisy Buttons](#)
- [Contact Bounce and Debouncing](#)